Experiment 9: A* Algorithm

Aim

To implement the A* search algorithm and find a least-cost path between a start node and a goal node using

path cost and heuristic cost.

Algorithm

1. Start

2. Initialize the open list with the start node.

3. Set g(n) as the path cost and h(n) as the heuristic estimate.

4. Calculate f(n) = g(n) + h(n).

5. Select the node with the smallest f(n).

6. Expand the selected node and update its neighbours.

7. Repeat until the goal node is selected.

8. Reconstruct and display the path.

9. Stop.

Flowchart

Start

↓

Initialize

↓

Process / Search

↓

Goal / Condition Reached?

n n

Yes No

↓ ↓

Print Result Repeat

↓

Stop

Objective

• Understand heuristic state-space search.

• Use f(n) = g(n) + h(n).

• Find an efficient path from a start node to a goal node.

Program

```
import heapq
def a_star(graph, heuristic, start, goal):
open_list = [(heuristic[start], 0, start)]
came_from = {}
g_cost = {start: 0}
while open_list:
f, current_g, current = heapq.heappop(open_list)
if current == goal:
path = [current]
while current in came_from:
current = came_from[current]
path.append(current)
return path[::-1], g_cost[goal]
for neighbour, cost in graph[current]:
new_g = g_cost[current] + cost
if neighbour not in g_cost or new_g < g_cost[neighbour]:
g_cost[neighbour] = new_g
came_from[neighbour] = current
new_f = new_g + heuristic[neighbour]
heapq.heappush(open_list, (new_f, new_g, neighbour))
graph = {
'A': [('B', 1), ('C', 4)],
'B': [('D', 2), ('E', 5)],
'C': [('E', 1)],
'D': [('F', 3)],
'E': [('F', 1)],
'F': []
```

```
}
heuristic = {'A': 5, 'B': 4, 'C': 2, 'D': 2, 'E': 1, 'F': 0}
path, cost = a_star(graph, heuristic, 'A', 'F')
print("A* Path:", " -> ".join(path))
print("Total Cost:", cost)
```

Output

```
A* Path: A -> B -> D -> F
Total Cost: 6
```

Result

The program was executed successfully and produced the expected result.

Conclusion

The experiment successfully demonstrated the corresponding Artificial Intelligence technique and its application